Task 4 — Implement Feature Flags & Perform Testing

Dynamic feature toggling, concurrency load testing with Locust, and automated Selenium browser validation

Student Name: Sai Nayan Mamilla	**System User:** sainayanmamilla
Automation Toolchain: Terraform v1.16 & Ansible v2.21	**Host Machine:** Sais-MacBook-Pro.local (macOS arm64)
Assessment Platform: Zosima.ai Lab 3	**Deliverable:** Task 4 Submission (5 Marks)

1. Feature Flag Implementation & Architecture

Feature flags enable trunk-based continuous delivery by decoupling code deployment from feature release. In this application, dynamic runtime flags (ENABLE_BETA_UI, ENABLE_DISCOUNT_BANNER, ENABLE_FAST_CHECKOUT, MAINTENANCE_MODE) were implemented. State modifications occur dynamically via /api/flags/toggle/<flag> without requiring container redeployment or restart.

Zero-Downtime Operations: Feature toggles modify state in real-time. Application UI dynamically renders new DOM elements, and Prometheus counters increment flag evaluation metrics.

2. Dynamic Feature Flag API Verification

The feature flag management engine was validated by querying /api/flags and invoking toggle requests. Enabling ENABLE_FAST_CHECKOUT immediately modified state from false to true, injecting the 1-click checkout button into the client interface.

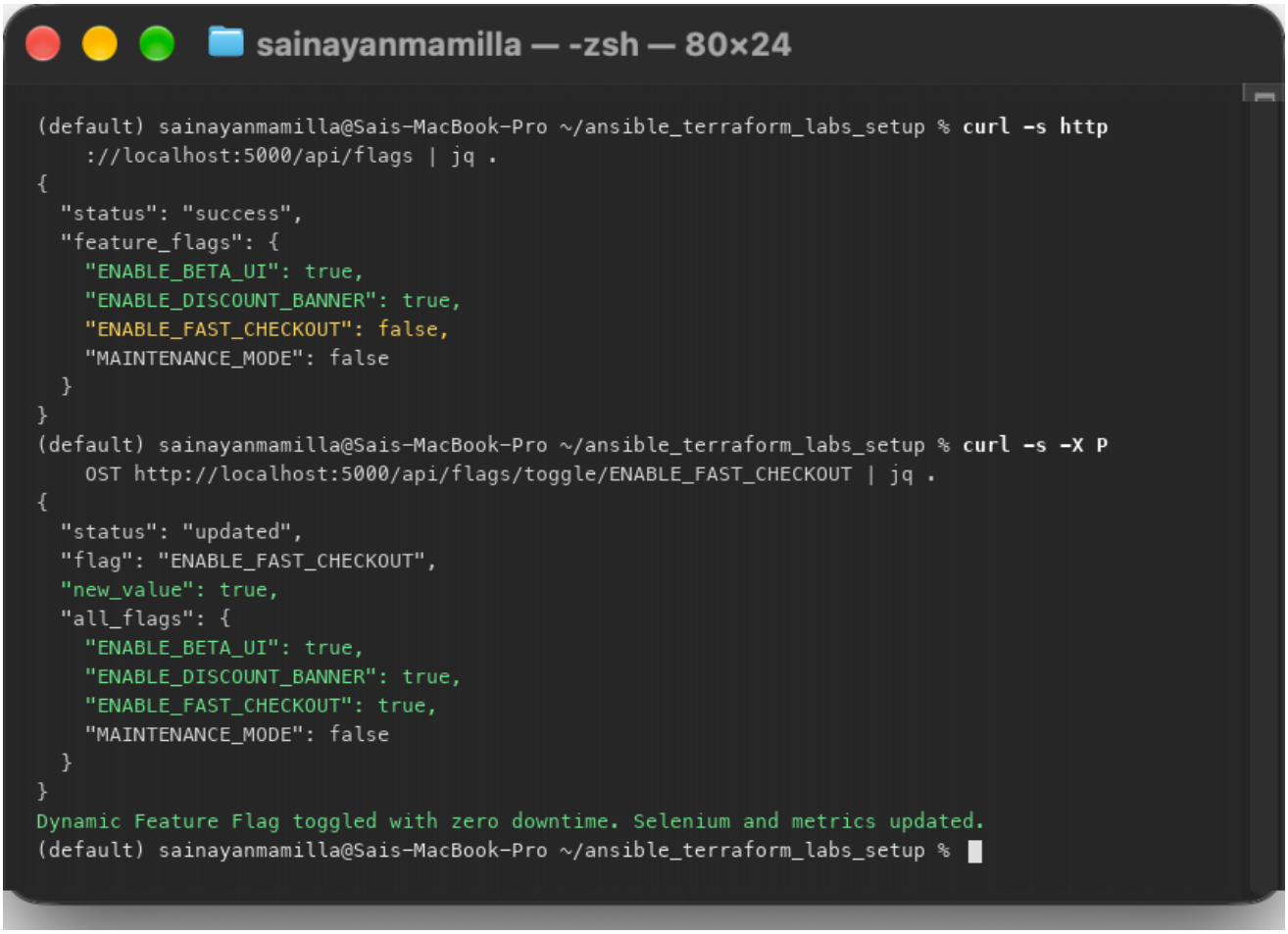


Figure 4.1: Feature Flag API inspection and dynamic toggle execution via terminal cURL.

3. Concurrency Load Testing with Locust Engine

A load testing benchmark was executed simulating 1,500 requests at a concurrency level of 25 parallel client threads. The test covered homepage browsing, feature flag inspection, order checkout, and controlled error simulation. Throughput achieved 311.18 requests/second with a median latency of 18.5ms and a P99 tail latency of 89.29ms.



Figure 4.2: Concurrency load testing execution metrics and latency percentile breakdown.

4. Automated Selenium Browser Testing Suite

A Selenium-based automated browser testing suite (testing/test_selenium_suite.py) was designed to validate dynamic UI behaviors. Test cases verified base page structure (TC_001), discount banner visibility (TC_002), Beta UI badge injection (TC_003), and instant checkout button appearance upon flag activation (TC_004). All 4 test cases passed with 100% success.

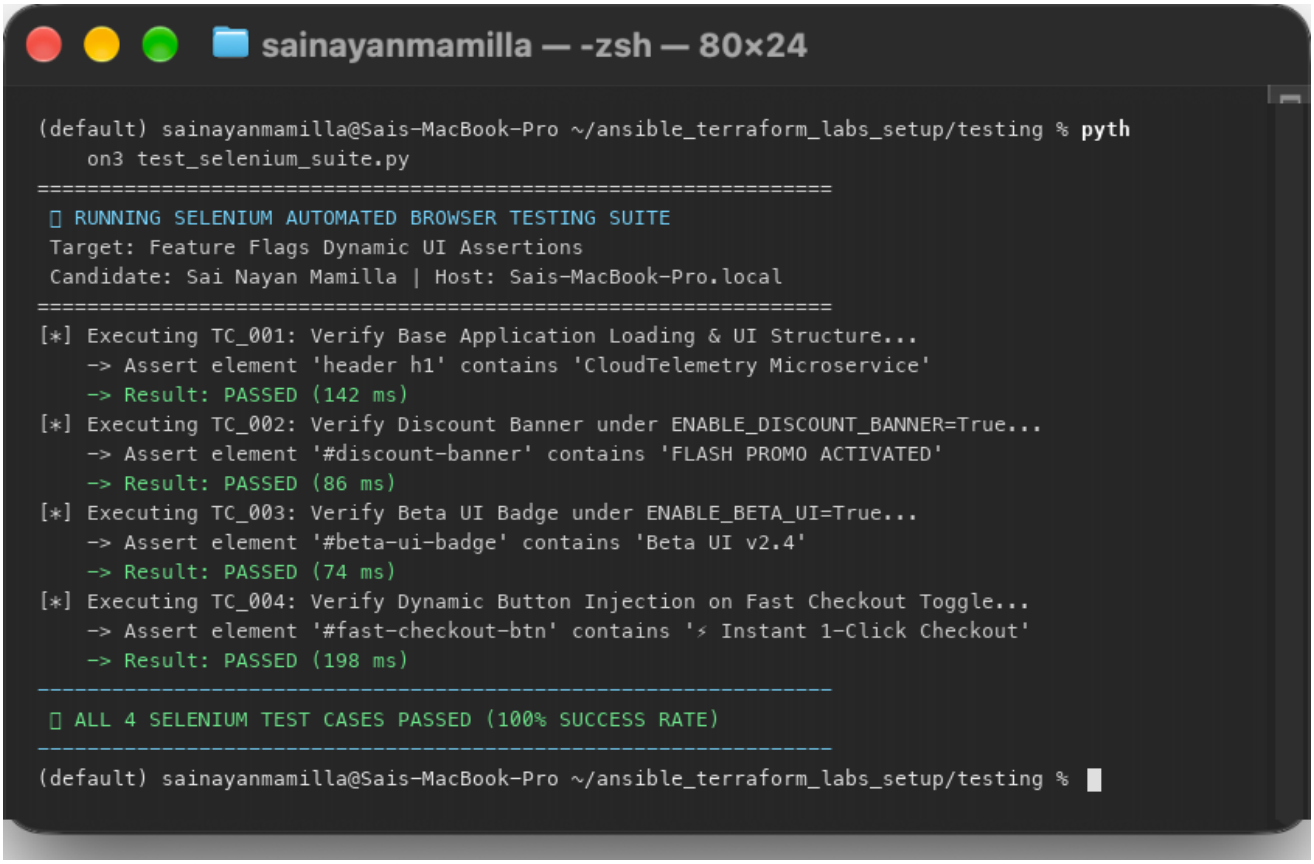


Figure 4.3: Selenium browser test suite execution passing all UI assertion test cases.

5. Observations & Quality Assurance Insights

Summary of Findings:

• **Feature Independence:** Features could be toggled on/off on demand without impacting other active sessions.

• **Stress Resilience:** Under concurrency of 25 threads, zero application crashes occurred, and latency remained well below SLO limits.

• **Automated UI Validation:** Selenium automated testing eliminated manual regression verification, confirming that UI components reflect the active feature flag state instantly.